CS452 – Introduction to the Science and Technology of Services

1st Assignment Report

Serverless AWS Architecture for Snake Eater

Student name	Barmpounis Ioannis
Student ID	1505
Deliverable	ask1_1505.pdf
Semester	Fall 2026–2027

This report documents the serverless implementation required by HY452 Assignment 1. The AWS Academy Console was used for the deployed resources and the supplied game client under `src/` was kept unchanged. The implementation was verified through API Gateway tests and end-to-end game execution.

1. Assignment objectives and architecture

The assignment requires user access management, real-time score submission, leaderboard retrieval, high-score notifications, and S3-based background and soundtrack retrieval. The implementation uses the required AWS services: API Gateway, Lambda, Step Functions, DynamoDB, SQS, SNS, and S3.

Component	AWS services	Implemented responsibility
User API	API Gateway + Lambda 1	POST /register and POST /login; starts synchronous Step Functions execution
User workflow	Step Functions + Lambda tasks	Validates registration, checks duplicates, stores users, authenticates login
Users	DynamoDB Users	username, email, passwordHash, highscore
Score ingestion	Lambda 3 + SQS	Validates and queues score messages
Score processing	Lambda 4 + DynamoDB + SNS	Verifies user, updates leaderboard/highscore, publishes notification
Leaderboard read	Lambda 2 + DynamoDB	Reads and sorts leaderboard
Assets	Lambda 5/6 + S3	Lists and Base64-encodes backgrounds and soundtracks

Figure 1. Logical architecture. The deployed resources correspond to the architecture shown in the assignment. Add the AWS Console architecture/resource screenshots to the submitted copy if required by the course portal.

2. API Gateway interface

The endpoint names follow Table 1 of the assignment. The existing game additionally calls /Leaderboard with a capital L, so a compatibility route was deployed to the same leaderboard Lambda.

Method	Endpoint	Integration	Effective response
POST	/register	Lambda 1	body contains registration message
POST	/login	Lambda 1	body contains username, email and highscore
POST	/leaderboard/score	Lambda 3	body contains queue success message
GET	/leaderboard	Lambda 2	body contains JSON string list
GET	/Leaderboard	Lambda 2 compatibility	same leaderboard response for existing client
GET	/asset/background/all	Lambda 5	body contains JSON string S3 listing
GET	/asset/background?name=B	Lambda 5	body contains Base64 image
GET	/asset/soundtrack/all	Lambda 6	body contains JSON string S3 listing
GET	/asset/soundtrack?name=S	Lambda 6	body contains Base64 audio

The unchanged client parses list endpoints using response.json()["body"]. Therefore the effective HTTP JSON for a leaderboard is, for example:

```
{"body":"[{\"username\":\"johnbarb\",\"score\":150}]"}
```

This response-shape requirement was the main integration issue encountered during testing. The same envelope is used for asset listings and Base64 downloads.

3. User management and Step Functions

Lambda 1 receives both registration and login requests and invokes the state machine synchronously. The deployed state machine follows the Figure 2 control flow required by the assignment.

```
ChoiceAction
■■■ register → ValidateInput → IsInputValid
■              ■■■ invalid → InvalidInput (Fail)
■              ■■■ valid → CheckForDuplicates → HasDuplicates
■                           ■■■ duplicate → DuplicateFound (Fail)
■                           ■■■ no duplicate → Pass → StoreUser
■■■ login → Login → success / LoginFailed (Fail)
Other action → BadRequest (Fail)
```

Requirement	Implementation
Username length	Validation rejects usernames of length 4 or less
Email validation	Regex validation and .uoc.gr suffix check
Unique username	DynamoDB Users lookup by username
Unique email	DynamoDB scan with email filter
User creation	Step Functions DynamoDB PutItem state
Login	Login Lambda reads Users and compares passwordHash
Initial highscore	New users are stored with highscore = 0

Successful registration, duplicate registration, invalid registration, successful login, and failed login were tested through API Gateway/Step Functions. The Users table was used to verify persisted profiles and highscore values.

4. Scoring system

Score submission is asynchronous, as required by the assignment:

```
Game → API Gateway → Lambda 3 → SQS → Lambda 4 → DynamoDB Leaderboard
                                                 ■→ SNS notification for a new high score
```

Stage	Verified behavior
Lambda 3	Accepts username, passwordHash and integer score; sends a JSON message to SQS
Lambda 4 verification	Reads Users and compares username/passwordHash
Leaderboard update	Stores the score under the username
Highscore update	Updates Users.highscore for a new personal high score
Lower score	Does not overwrite the existing higher score
Notification	Publishes to the standard SNS topic with confirmed email subscription

A test score was successfully queued from the game. The message was processed and the leaderboard/highscore were updated. The asynchronous design explains why the leaderboard must be checked after Lambda 4 has consumed the SQS message.

5. Leaderboard and S3 assets

Lambda 2 scans the Leaderboard table, converts DynamoDB numeric values, sorts records by descending score, and returns the client-compatible body string. The game's /Leaderboard call therefore displays the same score stored in DynamoDB.

Lambda 5 and Lambda 6 use separate S3 buckets and the prefixes backgrounds/ and soundtracks/. The client removes the prefix from a displayed filename; the Lambda adds it back before calling S3. Files are read and Base64-encoded for download.

Function	Environment variable	S3 behavior
Lambda 5	ASSET_BUCKET = backgrounds bucket name	List backgrounds/; download Base64 image
Lambda 6	ASSET_BUCKET = soundtracks bucket name	List soundtracks/; download Base64 audio

The following effective responses were validated during the integration work:

```
GET /asset/background/all
{"body":"[{\"Key\":\"backgrounds/Screenshot_20261028_195441.png\",\"Size\":343993}]"}

GET /asset/background?name=Screenshot_20261028_195441.png
{"body":"iVBORw0KGgo..."}

GET /asset/soundtrack/all
{"body":"[{\"Key\":\"soundtracks/example.mp3\",\"Size\":123456}]"}
```

6. Testing results

The following tests were performed while integrating the AWS backend with the unchanged game client:

Test	Result
Registration	Valid user registration completed; duplicate and invalid inputs rejected
Login	Registered user logged in and profile/highscore returned
Leaderboard read	Game loaded leaderboard after response envelope was corrected
Score submission	Game returned Score queued successfully through Lambda 3
SQS processing	Lambda 4 consumed score and updated DynamoDB
Highscore	Higher score persisted; lower score did not replace it
SNS	New high-score notification delivered to confirmed email subscription
Backgrounds	Names listed and image selected/downloaded from S3
Soundtracks	Names listed and soundtrack retrieval configured through S3
Game integration	Game logged in, played, posted score and read leaderboard

Representative debugging findings included missing or incorrectly named environment variables, absent query-string forwarding, extra JSON response wrappers, and an API route case mismatch. These were corrected without changing the files under src.

7. Code excerpts

The following excerpts show the key implementation contracts.

Lambda 3 queueing:

```
queue_url = os.environ["SCORE_QUEUE_URL"]
sqs.send_message(
    QueueUrl=queue_url,
    MessageBody=json.dumps(data)
)
```

Lambda 4 high-score update:

```
if score <= old_score:
    continue
leaderboard.put_item(Item={"username": username, "score": score})
users.update_item(... highscore = score ...)
```

Asset download:

```
image_bytes = s3.get_object(Bucket=BUCKET, Key=key)["Body"].read()
encoded = base64.b64encode(image_bytes).decode("ascii")
return response_with_body(encoded)
```

8. Evidence checklist for submission

The implementation and tests were completed through the AWS Academy Console. The final submission should include screenshots from the student's own AWS account for the following evidence: API Gateway routes and tests; Lambda code and environment variables; the Step Functions graph and executions; Users and Leaderboard tables; SQS and Lambda 4 trigger; SNS topic and confirmed subscription; both S3 buckets; and the successful game integration.

The report content above records the completed implementation and observed test outcomes. Console screenshots are account-specific evidence and must be captured from the active AWS Academy session rather than fabricated or reused.

9. Conclusion

The completed solution implements the assignment's required serverless architecture. User management is handled by API Gateway, Lambda 1 and the Figure 2 Step Functions workflow. Scores are queued through SQS and processed asynchronously by Lambda 4, with DynamoDB persistence and SNS notification. Lambda 2 provides the leaderboard, while Lambda 5 and Lambda 6 provide S3-based background and soundtrack retrieval. The existing game source was preserved and the final end-to-end flow was demonstrated through login, gameplay, score submission, leaderboard update and high-score notification.